

POC Puls-Events

Système de recommandation d'événements culturels

par génération augmentée par la recherche

Python 3.11

LangChain 0.3

Mistral AI

FAISS

Streamlit

Périmètre : Rennes, Bretagne · Juillet 2026

Comment trouver un événement culturel en langage naturel ?

- › Les moteurs de recherche par mots-clés ne comprennent pas l'intention de l'utilisateur
- › Un utilisateur veut demander : "Que faire à Rennes ce weekend en famille ?"
- › Il faut relier la compréhension du langage à des données événementielles réelles
- › Contrainte : stack open-source, environnement local reproductible

SOURCE DE DONNÉES

API Open Agenda — événements culturels Rennes / Bretagne, horizon 12 mois

OBJECTIF DU POC

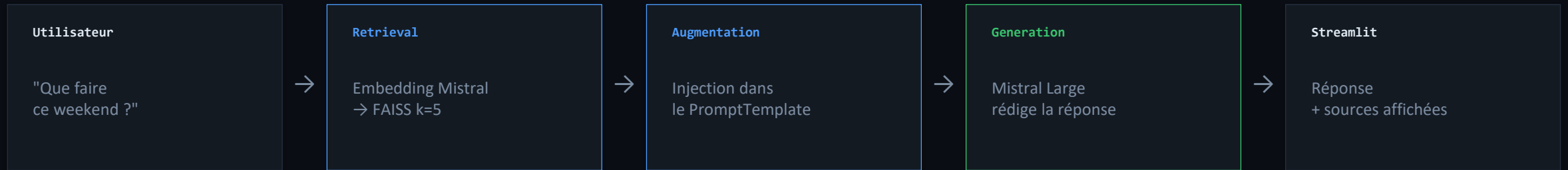
Valider la faisabilité d'un chatbot RAG sur un périmètre géographique réel, en 2 semaines

PÉRIMÈTRE CONFIGURABLE

La ville cible est paramétrable via `TARGET_CITY` dans le fichier `.env` — chaque équipe choisit sa ville

Retrieval-Augmented Generation

Un LLM seul ne connaît pas les événements locaux. Le RAG combine recherche sémantique et génération pour ancrer les réponses dans des données réelles.



Avantage clé : le modèle ne fabrique pas d'événements. Il ne répond qu'à partir des données indexées — si l'information est absente, il le signale explicitement.

Vue d'ensemble du système

PHASE OFFLINE — build_vectorstore.py



PHASE ONLINE — app.py (Streamlit)



STRUCTURE

src/ · data_collection/ · vectorstore/ · rag/ · evaluation/ · scripts/ · tests/

PRINCIPES

12 modules Python · une responsabilité par fonction · logging partout · config via .env

Les 5 briques du système

OPEN AGENDA API

Source de données événements culturels.
Pagination REST, filtre temporel intégré, réponse JSON.

MISTRAL AI

mistral-embed pour les vecteurs (1024 dim.).
mistral-large-latest pour la génération et le juge d'évaluation.

FAISS

Base vectorielle locale de Meta AI.
Recherche par similarité cosinus.
Aucun serveur requis — index sur disque.

LANGCHAIN

Orchestration RAG : RetrievalQA, PromptTemplate, Document, RecursiveCharacterTextSplitter.
v0.3.25

STREAMLIT

Interface chatbot web en pur Python.
@st.cache_resource — l'index FAISS est chargé une seule fois.
v1.45.1

Pipeline de données

- › `fetch_events()` — pagination jusqu'à 100 pages, filtre API `after[timings][gte]` = aujourd'hui – 365 jours
- › `is_within_one_year()` — vérifie chaque timing individuellement, ignore les dates malformées (`ValueError`)
- › `extract_fields()` — extrait : `id`, `title`, `description`, `city`, `address`, `first_date`, `url`
- › `clean_dataframe()` — supprime descriptions < 20 car., villes manquantes, lignes incomplètes
- › `preprocess()` — orchestrateur pur : charge → filtre → extrait → nettoie → sauvegarde

Principe : une fonction = une responsabilité. Chaque étape est testable et remplaçable indépendamment.

QUALITÉ DES DONNÉES

- Filtre temporel : événements < 1 an
- Filtre qualitatif : descriptions substantielles (≥ 20 car.)
- Encodage UTF-8 forcé

TESTS UNITAIRES (pytest)

`TestDataFreshness` · `TestGeographicFilter`
7 assertions vérifiant la fraîcheur, la complétude et la cohérence géographique des données

FORMAT DE SORTIE

`data/raw/events.json` — brut Open Agenda
`data/processed/events_clean.csv` — nettoyé, prêt pour FAISS

Construction de l'index FAISS

FAISS compare des vecteurs numériques. Mistral transforme chaque texte en un vecteur de 1 024 dimensions.

- › `build_documents()` — un Document LangChain par événement (titre + description)
- › `chunk_documents()` — découpe en blocs de 512 tokens, chevauchement 64 pour préserver le contexte
- › `build_index()` — vectorise via `FAISS.from_documents`, sauvegarde sur disque
- › `load_index()` — rechargement avec `allow_dangerous_deserialization=True` (pickle local)

POURQUOI CHUNKER ?

Un événement peut avoir une longue description. Découper garantit que la recherche retrouve le passage précis — pas tout le document.

POURQUOI 512 / 64 ?

512 tokens $\approx$ un paragraphe complet. Le chevauchement de 64 évite de couper une phrase entre deux chunks et de perdre du sens à la frontière.

INDEX SUR DISQUE

`data/processed/faiss_index/`
Rechargé une seule fois en mémoire via `@st.cache_resource` — pas de recalcul à chaque question.

La chaîne LangChain

- › Retriever : `vectorstore.as_retriever(k=5)` — 5 événements les plus proches sémantiquement
- › PromptTemplate : cadre le rôle, injecte le contexte, impose de ne répondre qu'à partir des données indexées
- › ChatMistralAI : `temperature=0.1` pour des réponses stables et factuelles
- › `chain_type='stuff'` : tous les documents injectés en une seule fois dans le prompt
- › `return_source_documents=True` : les sources sont renvoyées pour affichage dans l'interface

Si aucun événement pertinent n'est récupéré, le prompt demande au modèle de l'indiquer explicitement plutôt que d'inventer.

```
def build_rag_chain(vectorstore):  
    llm = ChatMistralAI(  
        model="mistral-large-latest",  
        temperature=0.1,  
    )  
    retriever = vectorstore.as_retriever(  
        search_kwargs={"k": 5}  
    )  
    return RetrievalQA.from_chain_type(  
        llm=llm,  
        retriever=retriever,  
        chain_type="stuff",  
        chain_type_kwargs={  
            "prompt": PROMPT_TEMPLATE  
        },  
        return_source_documents=True,  
    )
```


Chatbot Streamlit

- › `@st.cache_resource` — l'index FAISS et la chaîne ne sont instanciés qu'une seule fois
- › `st.session_state.messages` — persistance de l'historique de conversation entre les interactions
- › `st.chat_message` — bulles utilisateur / assistant distinctes
- › `st.chat_input` — champ de saisie ancré en bas de page
- › `st.expander` — sources affichées en accordéon (titre, date, ville, URL)

Lancement : `streamlit run app.py`
ou via : `python scripts/run_chatbot.py`

 Utilisateur

Que faire à Rennes ce weekend en famille ?

 Assistant

Ce weekend à Rennes, plusieurs activités sont disponibles pour les familles : Expo Sciences Vivantes au Musée de Bretagne (entrée gratuite le dimanche), Spectacle Marionnettes au TnB samedi à 15h...

▸ 3 sources • Musée de Bretagne • TnB • Opéra

LLM-as-a-judge

Évaluer un RAG manuellement est coûteux. On utilise Mistral lui-même comme juge automatique.

- › `generate_qa_pairs()` — Mistral génère 20 questions + réponses attendues à partir des événements réels
- › `evaluate()` — pour chaque question, on interroge le RAG, puis le juge compare les deux réponses
- › Le juge répond OUI (même sens) ou NON (informations différentes)
- › $\text{Score} = \text{nombre de OUI} / \text{total}$ · `temperature=0` pour un juge déterministe

Limite : le juge est le même modèle que le générateur. Pour un benchmark robuste : juge indépendant ou évaluation humaine.

PIPELINE D'ÉVALUATION

CSV événements → 20 paires Q/R générées → RAG répond → Juge Mistral → Score global

"Y a-t-il un concert de jazz ce mois ?"

OUI

"Musées ouverts dimanche à Rennes ?"

NON

"Expositions gratuites en juillet ?"

OUI

Ce que le POC démontre

100

pages max API

512

tokens / chunk

k=5

docs / requête

20

paires Q/R éval.

- › Réponses ancrées dans les données réelles — pas d'hallucination d'événements inexistant
- › Index chargé une seule fois en mémoire — temps de réponse stable
- › Pipeline entièrement reproductible : venv, .env, scripts documentés
- › Périmètre configurable — changer TARGET_CITY suffit
- › 7 tests unitaires vérifient fraîcheur et cohérence géographique
- › Architecture en couches — chaque module est indépendant et testable

Verdict POC : la stack LangChain + Mistral + FAISS est opérationnelle et validée sur des données réelles de l'API Open Agenda.

Limites du POC & Recommandations

LIMITES ACTUELLES

- › Index statique — rebuild manuel dès que de nouveaux événements arrivent
- › Évaluation sur 20 Q/R — insuffisant pour un scoring de production fiable
- › Single-tenant — une seule ville configurée à la fois
- › Pas d'authentification ni de persistance de l'historique entre sessions
- › Dépendance API Mistral — coût à l'usage, latence réseau

POUR LA VERSION DE PRODUCTION

- › Mise à jour incrémentale de l'index (FAISS.add_documents ou Qdrant / Pinecone)
- › Déploiement cloud — Streamlit Community Cloud, AWS ou GCP
- › Support multi-villes avec sélecteur dans l'interface Streamlit
- › Évaluation continue avec dataset de référence croissant (> 100 Q/R)
- › Monitoring : coût API, latence, qualité des réponses, drift des données
- › Authentification et persistance de l'historique en base de données